



S72579: Going Deeper with Multi-GPU Scaling: Task-based Runtimes

Manolis Papadakis, Software Engineering Manager, NVIDIA

Mar 20, 2025



Motivation

Data Model

Interoperability

Conclusion

When Do You Need Multiple GPUs?

- Simply when you need more than what a single GPU can do
- Example scenarios
 - Having lots of computation that a single GPU is insufficient to perform (e.g., DL training)
 - Processing a large volume of data that can benefit from a large aggregate memory bandwidth

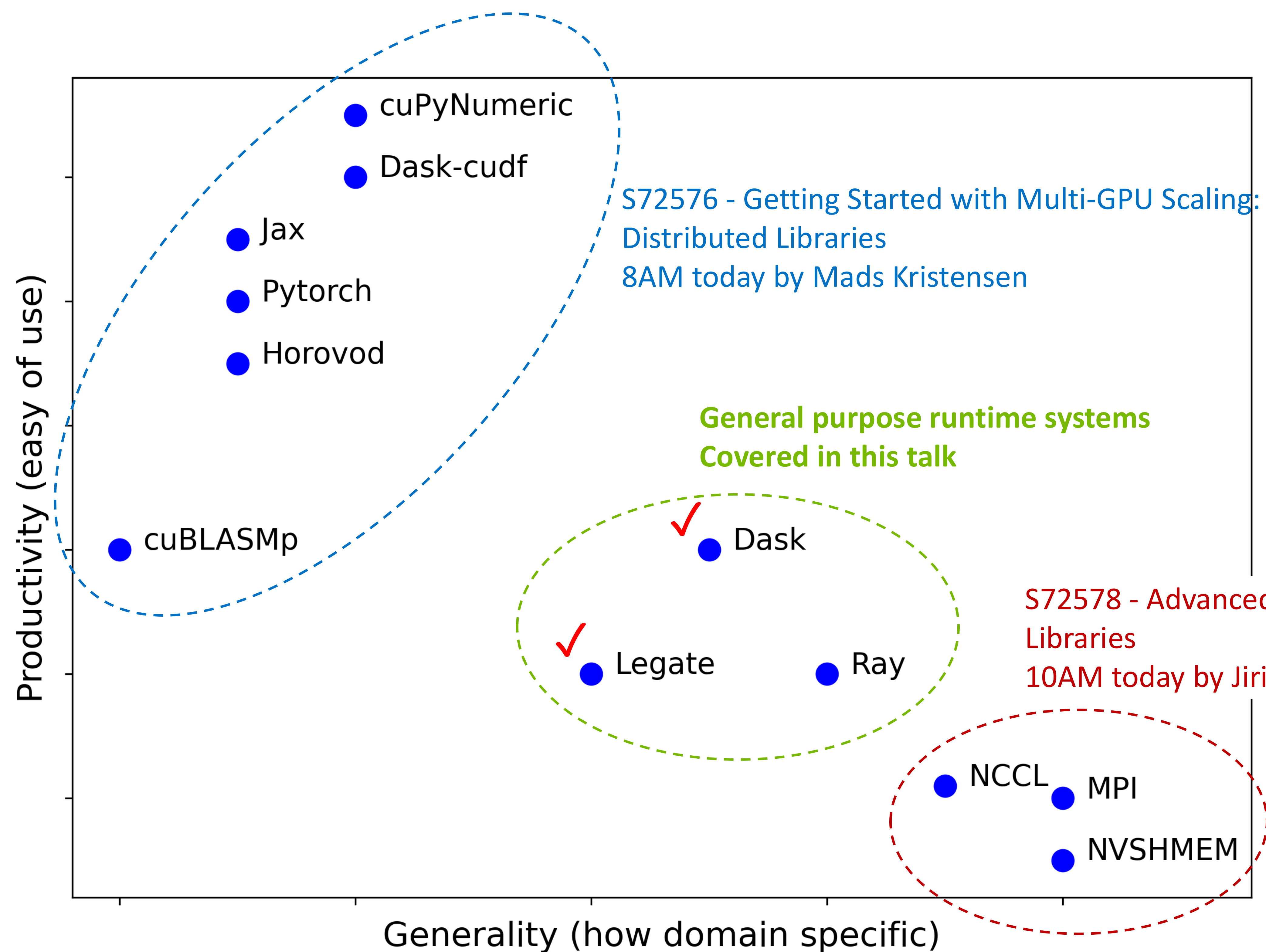
Why is Multi-GPU Programming Hard?

In addition to the usual difficulties with single GPU programming, multi-GPU systems require you to:

- Partition computation and data to utilize all the GPUs
- Handle data movement between different GPU memories
- Coordinate/synchronize execution across GPUs for any dependent computation

Getting a functionally correct multi-GPU code can be challenging, let alone perf-optimizing the code

Multi-GPU Programming Landscape



Task-Based Paradigm

- Program = a bag of tasks
 - Tasks are a quantum of execution, each running on a single GPU
 - Runtime is responsible for scheduling, synchronization, data transfers
- Parallelism = independent tasks
 - Runtime is responsible for finding (data) dependencies between tasks
 - Programmers are responsible for enabling parallel execution, by providing independent tasks
- Benefits
 - Simple programming model:
 - User code operates at the “logical” level (one object representing the entire dataset, split across memories)
 - Runtime distributes the work across the GPUs
 - Deadlock & data-race freedom
 - Facilitate sequential reasoning on the program semantics

Legate and Dask

- Two representative task-based runtimes with interesting design choices
- Will compare features using simple examples
 - Focusing on features relevant to (multi-)GPU execution
 - Assuming a 2-GPU machine
 - Deliberately not using higher-level APIs
- Full examples available at <https://github.com/manopapad/legate-vs-dask>



Legate	Dask
HPC origin	Python/PyData focused
Both Python and C++ APIs	Python only
Native data abstractions (white-box approach)	Treating library data abstractions as a black box
Deferred execution	Lazy execution
Multi-controller model	Single-controller model (client-server architecture)
Implicit task graph building	Explicit task graph building
Mutable data abstractions	Immutable data abstractions

Tasking Model

- Both runtimes offer a primitive to turn Python functions into tasks.

```
from legate.core.task import task
```

```
@task  
def hello():  
    print("Hello, world")
```

```
hello()
```

Legate

```
import dask
```

```
@dask.delayed  
def hello():  
    print("Hello, world")
```

```
hello.compute()()
```

Dask

Deferred execution model

- Task launches are implicit
- Execution is not immediate but will happen eventually

Lazy execution model

- Task launches are explicit (.compute(), .persist(), etc.)
- Execution is guaranteed only when requested



Motivation

Data Model

Interoperability

Conclusion

Data Partitioning and Parallelism

- Both runtimes extract parallelism by partitioning data and launching independent tasks, one for each piece.

```
@task(variants=(GPU,),
      constraints=(
          # partition these in the same way
          align("dst", "src"),
          # don't split the first dimension
          broadcast("src", (0,))))
def batched_fft(
    dst: OutputStore, src: InputStore,
):
    cp_dst = cupy.asarray(dst)
    cp_src = cupy.asarray(src)
    cp_dst[:] = cupy.fft.fftn(cp_src, axes=(0,))

    shape = (1024, 1024)
    x = runtime.create_store(complex64, shape)
    y = runtime.create_store(complex64, shape)
    runtime.issue_fill(y, 42)

    batched_fft(x, y)
```

Legate

```
def batched_fft(src: cupy.ndarray) -> cupy.ndarray:
    return cupy.fft.fftn(src, axes=(0,))

shape = (1024, 1024)
# manually make sure we don't split the first dim
chunks = (1024, 512)

with dask.config.set({"array.backend": "cupy"}):
    x = dask.array.full(
        shape, 42, chunks=chunks, dtype=complex64
    )

    y = x.map_blocks(batched_fft)
    wait(y.persist())
```

Dask

- Automatic partitioning by default: Runtime controls the data split, and degree of parallelism (e.g. based on data sizes, subsequent operations).
- Enabled by "scale-free" tasks (task bodies must be agnostic to the actual degree of parallelism).
- Instead, tasks must describe desirable shapes of partitions via partitioning constraints.

- Manual partitioning by default: user has to select chunk size.

White-Box vs. Black-Box Partitioning

Legate (white-box):

- High-level collections are built on top of "global" data abstractions provided by the runtime.
- Runtime has visibility over the global decomposition of data.
- Stores are mutable. Tasks signatures describe access modes (e.g. Input vs Output), runtime guarantees data coherence in accordance to these.
- One store can be partitioned in multiple ways. Partitions will be kept in sync by the runtime.

Dask (black-box):

- High-level collections are bags of "local" futures pointing to per-GPU chunks.
- Runtime has no inherent knowledge of where each chunk belongs within the global dataset. The library needs to keep track of these semantics.
- Collections are immutable.
- Each (copy of) the collection has one partition.

Repartitioning

- What higher-level libraries (e.g. cuPyNumeric, `dask.array`) need to do to change data partitioning.
- Showing row-wise to column-wise shuffle, more complicated cases include reduction trees, stencils, CSR matrices, string-type dataframes.

```
@task(variants=(GPU,),
      constraints=(broadcast("x", (1,)),))
def row_wise(x: InputStore) -> None:
    assert cupy.asarray(x).shape == (500, 3000)

@task(variants=(GPU,),
      constraints=(broadcast("x", (0,)),))
def col_wise(x: InputStore) -> None:
    assert cupy.asarray(x).shape == (1000, 1500)

store = runtime.create_store(int8, (1000, 3000))
runtime.issue_fill(store, 1)

# Store starts out row-partitioned
row_wise(store)

# Legate recognizes the store should be repartitioned
col_wise(store)
```

Legate

```
# dask.array.ones((1000, 3000), chunks=(500, 3000))
row_wise = (
    dask.delayed(cupy.ones)((500, 3000)),
    dask.delayed(cupy.ones)((500, 3000)),
)

# dask.array.rechunk(row_wise, (1000, 3000))
top = dask.delayed(cupy.hsplit)(row_wise[0], 2)
top_l = dask.delayed(lambda x: x[0])(top)
top_r = dask.delayed(lambda x: x[1])(top)
bot = dask.delayed(cupy.hsplit)(row_wise[1], 2)
bot_l = dask.delayed(lambda x: x[0])(bot)
bot_r = dask.delayed(lambda x: x[1])(bot)
col_wise = (
    dask.delayed(cupy.vstack)((top_l, bot_l)),
    dask.delayed(cupy.vstack)((top_r, bot_r)),
)
```

Dask

- Repartition per the needs of the operation.
- Works even when moving data between libraries.
- Manually build up the graph to execute the shuffle.
- Runtime doesn't know what piece each future is meant to represent, so it can't check your work.

Mixed CPU-GPU Execution

```
@task(variants=(CPU,))
def initialize(dst: OutputStore) -> None:
    numpy.asarray(dst)[:]= 1

shape = (2**31,)
x = runtime.create_store(int8, shape)

# task can only run on CPU
# so the Store starts out on RAM
initialize(x)

# cuPyNumeric can handle both CPU and GPU execution
# but we explicitly force GPU execution
# Legate emits (potentially inter-node) h2d copies
with machine.only(TaskTarget.GPU):
    y = cupynumeric.add(x, 1)
```

Legate

- Runtime keeps track of (potentially multiple) instances of the same data, on CPU and GPU memory.
- Emits copies and invalidations as necessary.
- Executes the copies using a DMA path planner.

```
shape = (2**31,)
chunks = (2**30,)

# starts out as collection of NumPy chunks, on RAM
x = dask.array.ones(shape, chunks=chunks, dtype=int8)

# move each chunk to GPU memory, in parallel
y = x.map_blocks(cupy.asarray)

# back to NumPy
z = y.map_blocks(cupy.asnumpy)

assert(z.max().compute() == 1)
```

Dask

- Runtime is agnostic to where each piece of a collection resides, it treats the collection as a bag of futures.
- Library is responsible for moving data between memories on a per-chunk basis.
- Relies on Dask-CUDA and UCX-Py to perform transfers efficiently, using the accelerated network, depending on the source & target memories.



Motivation

Data Model

Interoperability

Conclusion

Interoperability Between Downstream Libraries

```
shape = (1024,)
ary = cupynumeric.ones(shape, int32)

# in this case, no repartitioning needs to happen
col = LogicalColumn(ary)
print(reduce(col, aggregation.sum(), int32)) # 1024

# the column and array refer to the same data
# whether or not repartitioning needed to happen
cupynumeric.add(ary, 1, out=ary)
print(reduce(col, aggregation.sum(), int32)) # 2048
```

Legate

```
shape = (1024,)
ary = dask.array.ones(shape, dtype=int32)

# in this case, no repartitioning needs to happen
# but the series is a copy of the original array
series = dask.dataframe.from_dask_array(ary)
assert series.sum().compute() == 1024
```

Dask

- cuPyNumeric and Legate-Dataframe both build on top of Legate Arrays.
- Based on the partitioning constraints the libraries declare, Legate knows if it needs to repartition when going between them.
- The libraries don't need to know anything about each other.
- Dask-Dataframe knows the internals of Dask-Array, so it can figure out the array's partitioning, to build a task graph to convert from it.
- Each collection must know the internals of any other that it wants to import from / export to.

Interoperability With Collective Communication Libraries

```
static inline const auto GPU_VARIANT_OPTIONS =
    lg::VariantOptions{}.with_communicators({"nccl"});

static void gpu_variant(lg::TaskContext context)
{
    // Communicators cached by runtime
    // reused when using the same devices
    auto comm =
        context.communicators().at(0).get<ncclComm_t*>();

    auto output = context.output(0);
    auto shape = output.shape<1>();
    auto volume = shape.volume();
    auto ptr =
        output.data().write_accessor<uint64_t, 1>()
            .ptr(shape.lo);

    // Streams managed by the runtime
    // for tying CUDA effects into task completion
    auto stream = context.get_task_stream();
    CHECK_NCCL(
        ncclAllReduce(ptr, ptr, volume,
                      ncclUint64, ncclSum, *comm, stream));
}
```

Legate

- Useful for algorithms requiring all-to-all communication or fine-grained communication/computation overlap.
- Legate ensures that workers taking part in the same collective execute concurrently, and atomically vs other tasks doing collectives over the same GPUs. This may be necessary for deadlock safety.
- Dask can also launch workers that internally do collective communication, but default scheduler offers no concurrent execution support.
- But consider Actor support in Dask, and the Explicit Comms module in Dask-CUDA).
- For more information on using collective communication libraries effectively, stay in the room for the **10AM talk: S72578 - Advanced Multi-GPU Scaling: Communication Libraries**, by Jiri Kraus

Interoperability Between Tasking Runtimes

- E.g. using Legate and Dask in the same program
- Not straightforward, as the internal representations of computation are not composable between runtimes.
- If even possible, requires the runtimes to:
 - Provide a well-defined import/export schema for partitioned data
 - Copy data, if the partitioning schemes are incompatible
 - Transfer ownership of data if not copied
 - Coordinate resource allocation (e.g. memory)
 - At handoff, complete all outstanding operations, clear any intermediates, then pass control to the other
- Expensive operation, not to be done frequently
- No standard approach, typically done on a case-by-case basis
- No native support in either runtime



Motivation

Data Model

Interoperability

Conclusion

Concluding Remarks

- Legate and Dask are both task-based runtimes, that take different approaches in certain key areas.
 - Primarily white-box vs black-box data abstractions
- Domain libraries can be built over both, but better abstractions at the runtime level means that downstream libraries (and user codes that combine them) don't need to:
 - Maintain global data structure invariants manually.
 - Manage data coherence, potentially across CPU and GPU instances of the same data.
 - Pick the right partition size
 - Know the internals of other collections to interoperate with them.
 - Decide when it's necessary (or beneficial) to repartition.
 - Implement complicated repartitioning patterns.
 - Ensure deadlock safety when using communicators.
 - Lose out on memory reuse potential offered by mutable data structures.

CUDA Developer Sessions

General CUDA

[S72571](#) - What's CUDA All About Anyways?

[S72897](#) - How To Write A CUDA Program: The Parallel Programming Edition

CUDA Python

[S72450](#) - Accelerated Python: Tour of the Community and Ecosystem

[S72448](#) - The CUDA Python Developer's Toolbox

[S72449](#) - 1001 Ways to Write CUDA Kernels in Python

[S74639](#) - Enable Tensor Core Programming in Python With CuTe

CUDA C++

[S72574](#) - Building CUDA Software at the Speed-of-Light

[S72572](#) - The CUDA C++ Developer's Toolbox

[S72575](#) - How You Should Write a CUDA C++ Kernel

Developer Tools

[S72527](#) - It's Easier than You Think – Debugging and Optimizing CUDA with Intelligent Developer Tools

Connect with the Experts

[CWE72433](#) – CUDA Developer Best Practices

[CWE73310](#) - Using NVIDIA CUDA Compiler Tool Chain for Productive GPGPU Programming

[CWE72393](#) - What's in Your Developer Toolbox? CUDA and Graphics Profiling, Optimization, and Debugging Tools

Multi-GPU Programming

[S72576](#) - Getting Started with Multi-GPU Scaling: Distributed Libraries

[S72579](#) - Going Deeper with Multi-GPU Scaling: Task-based Runtimes

[S72578](#) - Advanced Multi-GPU Scaling: Communication Libraries

Performance Optimization

[S72683](#) - CUDA Techniques to Maximize Memory Bandwidth and Hide Latency

[S72685](#) - CUDA Techniques to Maximize Compute and Instruction Throughput

[S72686](#) - CUDA Techniques to Maximize Concurrency and System Utilization

[S72687](#) - Get the Most Performance from Grace Hopper



nvidia.com/gtc/sessions/cuda-developer



Control Over Task Placement

- Both runtimes have heuristics for deciding where to place work.
- Primary consideration is minimizing data movement.
- Both allow user overrides.

```
@task(variants=(GPU,))
def increment(
    dst: OutputStore, src: InputStore
) -> None:
    cupy.asarray(dst)[:]= cupy.asarray(src) + 1

shape = (2**31,)
x = runtime.create_store(int8, shape)
runtime.issue_fill(x, 1)

# full data is pulled to GPU 0, work happens there
with machine.only(TaskTarget.GPU)[0]:
    increment(x, x)

assert(cupynumeric.asarray(x).max() == 2)
```

Legate

```
# one worker per GPU
w0, w1 = client.scheduler_info()['workers'].keys()

with dask.config.set({"array.backend": "cupy"}):
    # data is initialized in two chunks
    x = dask.array.ones(
        (2**21,), chunks=(2**20,), dtype=int8
    )

# full data is pulled to GPU 0, work happens there
with dask.annotate(workers=(w0,)):
    y = x + 1

assert(z.max().compute() == 2)
```

Dask

Control Over Memory Use

- Remote data is collectable once all client references are removed, and it's not needed by outstanding tasks

```
@task(variants=(GPU,))
def increment(
    dst: OutputStore, src: InputStore
) -> None:
    cupy.asarray(dst)[:]= cupy.asarray(src) + 1

shape = (2**31,)
x = runtime.create_store(int8, shape)
runtime.issue_fill(x, 42)
y = runtime.create_store(int8, shape)
z = runtime.create_store(int8, shape)

# intermediate values are materialized
# but obey typical reference counting rules
increment(y, x)
del x

# can also force offload to another memory
increment(z, y)
y.offload_to(SYSMEM)

# in-place updates save on intermediates
increment(z, z)
```

Legate

```
def increment(arr: cupy.ndarray) -> cupy.ndarray:
    return arr + 1

shape = (2**31,)
chunks = (2**30,)

with dask.config.set({"array.backend": "cupy"}):
    x = dask.array.full(
        shape, 42, chunks=chunks, dtype=int8
    )

# only the persisted values are kept around
# refs to non-persisted values don't matter
y = x.map_blocks(increment)
z = y.map_blocks(increment)
wait(z.persist())

# dask-cuda heuristically spills objects to RAM
# when memory utilization is high
```

Dask

Interoperability With Single-GPU Libraries

- Already seen use of single-GPU libraries inside tasks (tasks have full visibility to their local pieces, including pointers that can be passed through e.g. CUDA Array Interface)
- Can also move data between top-level code and remote-running tasks (note that doing work at the top-level blocks task launching, so is not recommended).

```
@task(variants=(GPU,))
def initialize(dst: OutputStore) -> None:
    cupy.asarray(dst)[:]= 1

@task(variants=(GPU,))
def increment(
    dst: OutputStore, src: InputStore
) -> None:
    cupy.asarray(dst)[:]= cupy.asarray(src) + 1

# initialize data scattered across GPUs
x = runtime.create_store(int8, (2**20,))
initialize(x)

# pull the data onto first local GPU memory
y = cupy.asarray(x)
assert y.sum() == 2**20

# remote data and local view are kept in sync
increment(x, x)
assert y.sum() == 2**21
```

Legate

```
# array instantiated on the first local GPU
x = cupy.arange(2**21, dtype=int32)

# tiled and scattered across the cluster's 2 GPUs
# Dask-CUDA recognizes that source and destination
# are both GPU memories, and uses NVLink/GPUDirectRDMA
chunks = (2**20,)
with dask.config.set({"array.backend": "cupy"}):
    y = dask.array.from_array(x, chunks)

# force the data back onto local GPU memory
z = y.compute()
assert cupy.all(x == z)
```

Dask

Legate: Creating Views to Arrays

Tasks can access arrays using views

```
@task(variants=(CPU,GPU))
def fill(
    to_fill: OutputStore, val: int
) -> None:
    try:
        arr = numpy.asarray(to_fill)
    except ValueError:
        arr = cupy.asarray(to_fill)
    arr[:] = val

array = runtime.create_array(int64, (9, 9))

fill(array, 0)

fill(array[1:-1, 1:-1], 1)

fill(array[1:-1, 1:-1][1:-1, 1:-1], 2)

assert numpy.sum(array.get_physical_array()) == 74
```

Legate

Creates a task for both CPU and GPU execution

Physical arrays conditionally offer the array interface and the CUDA array interface depending on where they are allocated

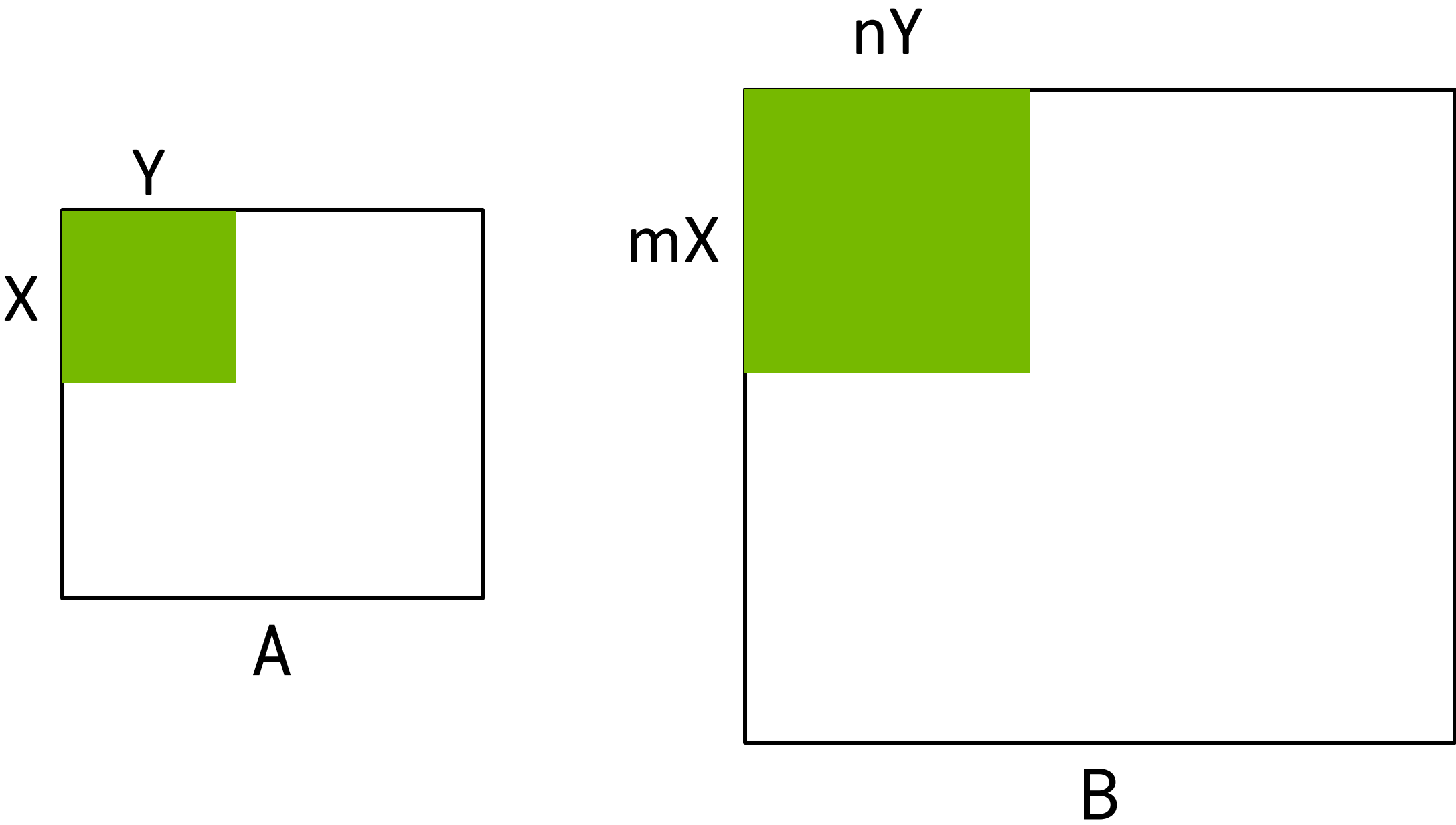
A view to the interior is passed to the task

Views can be nested

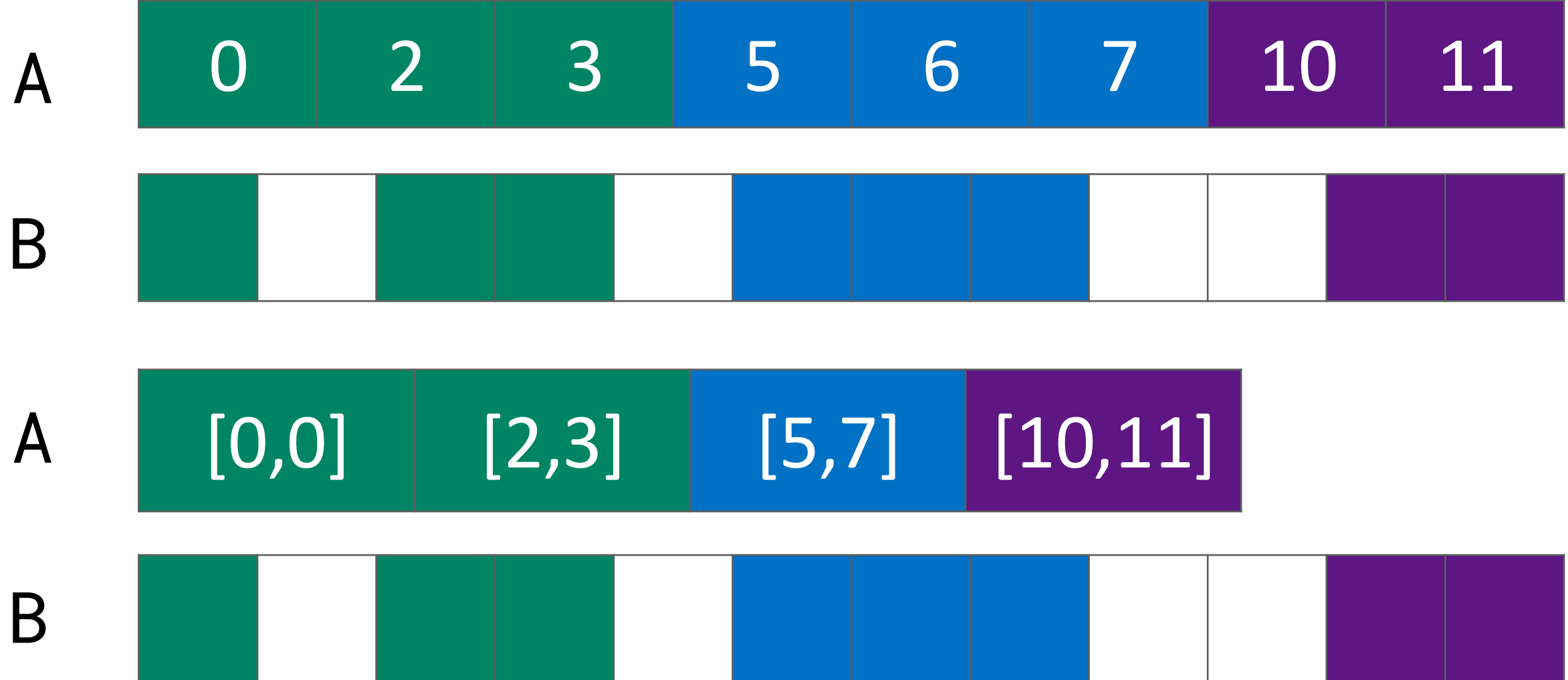
Gathers `array`'s entire data on the host memory

Partitioning Constraints in Legate

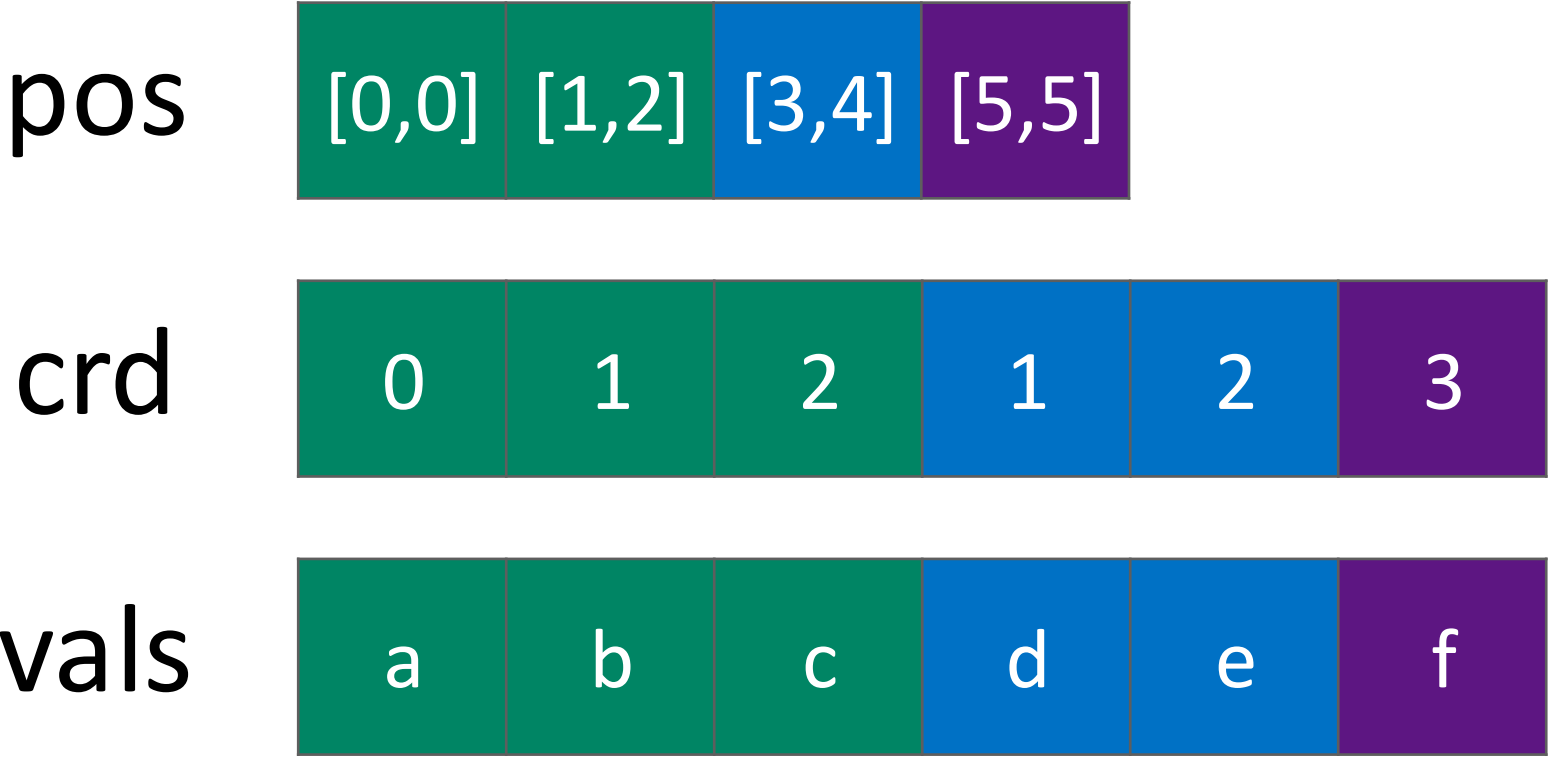
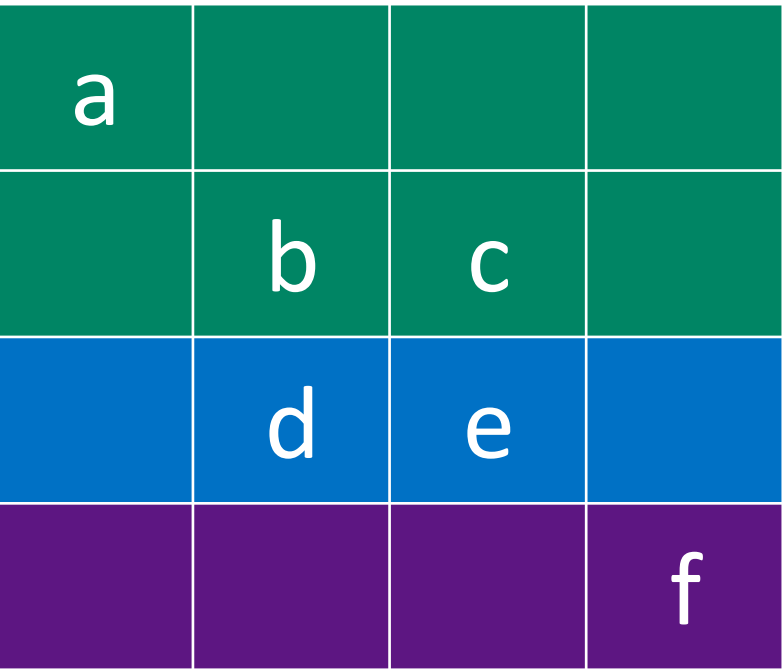
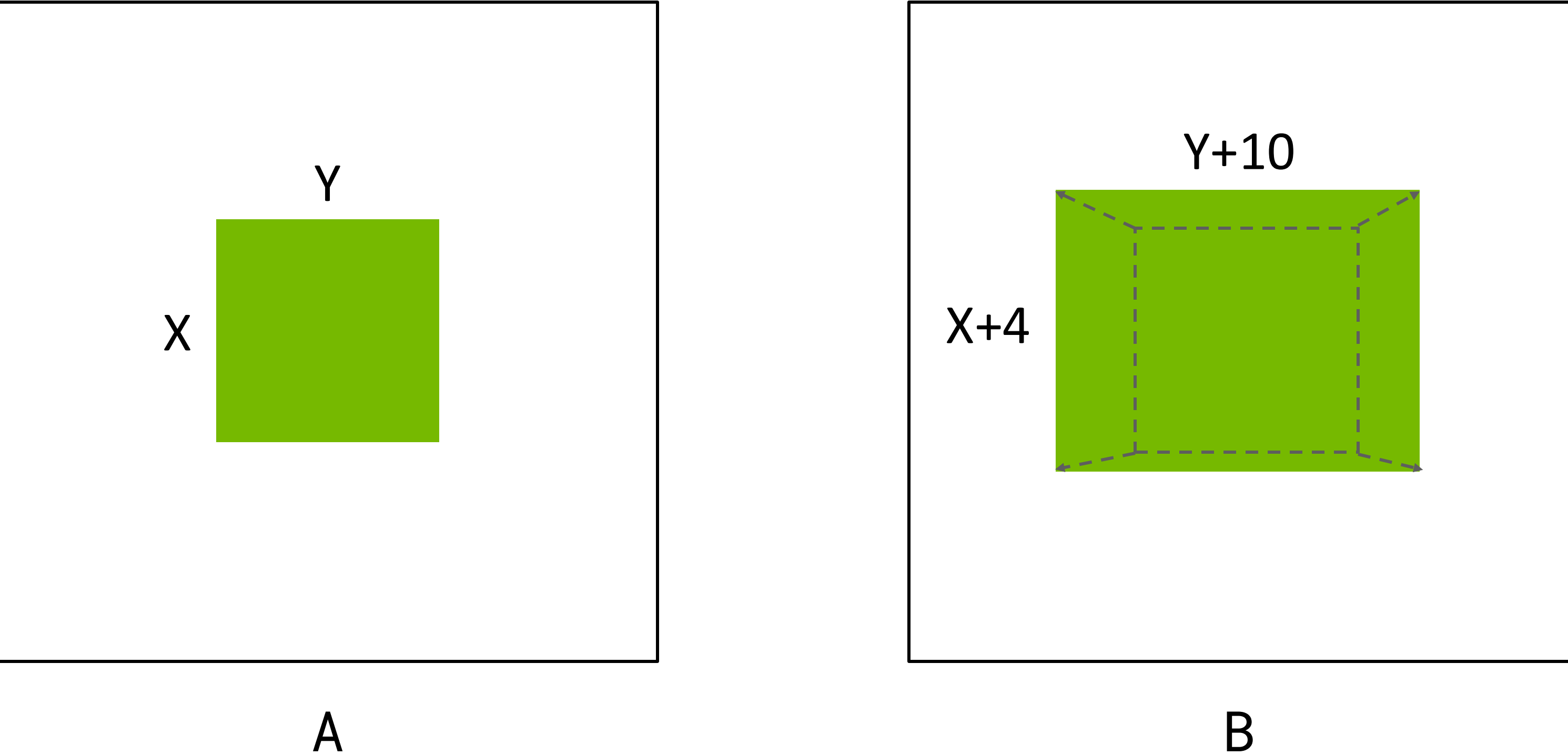
scale((m,n), A, B)



image(A, B)



bloat(A, B, low_offsets=(3,6), high_offsets=(1,4))



CSR matrix partitioned by row

Legate: Manual Task Launch

Example: GEMM for Cholesky Factorization

```

    Retrieve a sub-store by name
    ↓
rhs2 = p_output.get_child_store(k, i)
lhs = p_output
rhs1 = p_output

launch_domain = Rect(lo=(lo, k), hi=(hi, k + 1))
task = context.create_task(
    CuNumericOpCode.GEMM, manual=True, launch_domain=launch_domain
)
task.add_output(lhs)
task.add_input(rhs1, proj=lambda p: (p[0], i))
task.add_input(rhs2)
task.add_input(lhs)
task.execute()

```

Store partition argument

Launch domain

Custom projection function

Pseudocode:

for (y, x) = (lo, k), (hi, k + 1):

 # lhs[y][x] is inout

 GEMM(lhs[y][x], rhs1[y][i], rhs2[k][i])

